



Sistemas de Operação

Memory management

Artur Pereira <artur@ua.pt>

DETI / Universidade de Aveiro

Outline

- ① Introduction
- ② Address space of a process
- ③ Analysing the logical address space of a process
- ④ Contiguous memory allocation
- ⑤ Memory partitioning
- ⑥ Virtual memory system
- ⑦ Paging
- ⑧ Segmentation
- ⑨ Page replacement
- ⑩ Bibliography

Memory management

Introduction

- To be executed, a process must have its address space, at least partially, resident in main memory
- In a multiprogrammed environment, to maximize processor utilization and improve response time (or turnaround time), a computer system must maintain the address spaces of multiple processes resident in main memory
- But, there may not be room for all
 - because, although the main memory has been growing over the years, it is a fact that “data expands to fill the space available for storage”

(Corollary of the Parkinson's law)

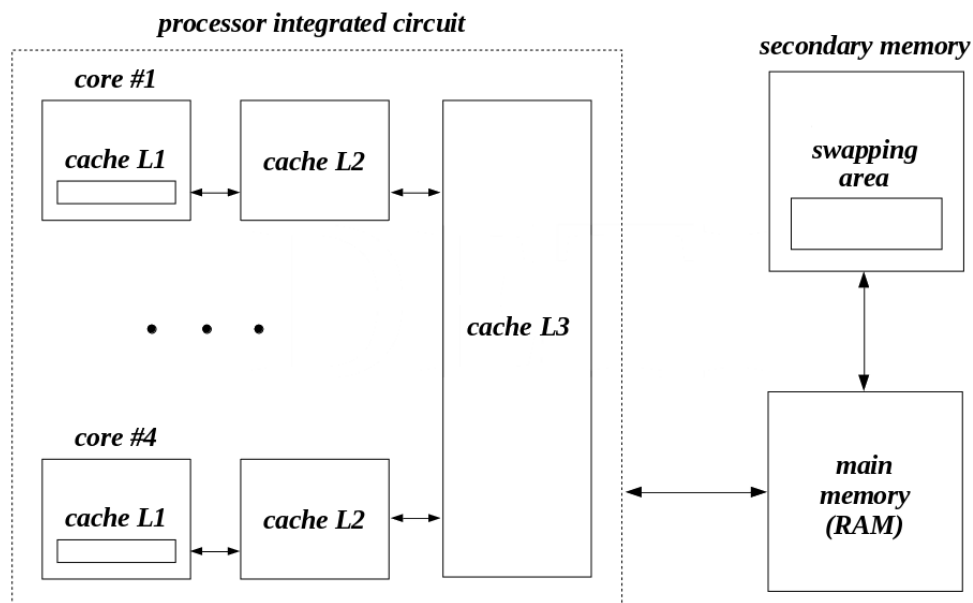
Memory management

Memory hierarchy

- Ideally, an application programmer would like to have infinitely large, infinitely fast, non-volatile and inexpensive memory available
 - In practice, this is not possible
- Thus, the memory of a computer system is typically organized at different levels, forming a hierarchy
 - **cache memory** – small (tens of KB to some MB), very fast, volatile and expensive
 - **main memory** – medium size (hundreds of MB to hundreds of GB), volatile and medium price and medium access speed
 - **secondary memory** – large (tens, hundreds or thousands of GB), slow, non-volatile and cheap

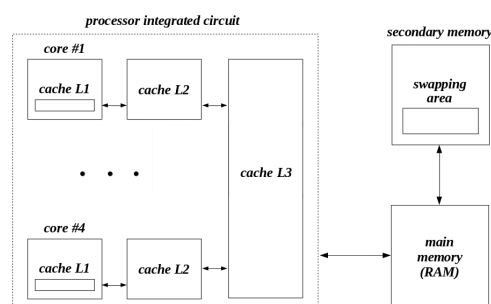
Memory management

Memory hierarchy (2)



Memory management

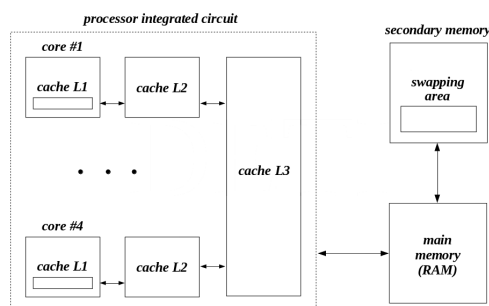
Memory hierarchy (3)



- The **cache memory** will contain a copy of the memory positions (instructions and operands) most frequently referenced by the processor in the near past
 - The cache memory is located on the processor's own integrated circuit (**level 1**)
 - And on an autonomous integrated circuit glued to the same substrate (**levels 2 and 3**)
 - Data transfer to and from main memory is done almost completely transparent to the system programmer

Memory management

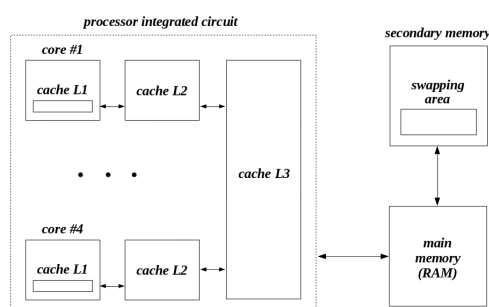
Memory hierarchy (3)



- **Secondary memory** has two main functions
 - **File system** – storage for more or less permanent information (programs and data)
 - **Swapping area** – Extension of the main memory so that its size does not constitute a limiting factor to the number of processes that may currently coexist
 - the swapping area can be on a disk partition used only for that purpose or be a file in a file system

Memory management

Memory hierarchy (4)



- This type of organization is based on the assumption that the further an instruction or operand is away from the processor, the less times it will be referenced
 - In these conditions, the mean time for a reference tends to be close to the lowest value
- Based on the **principle of locality of reference**
 - The tendency of a program to access the same set of memory locations repetitively over a short period of time

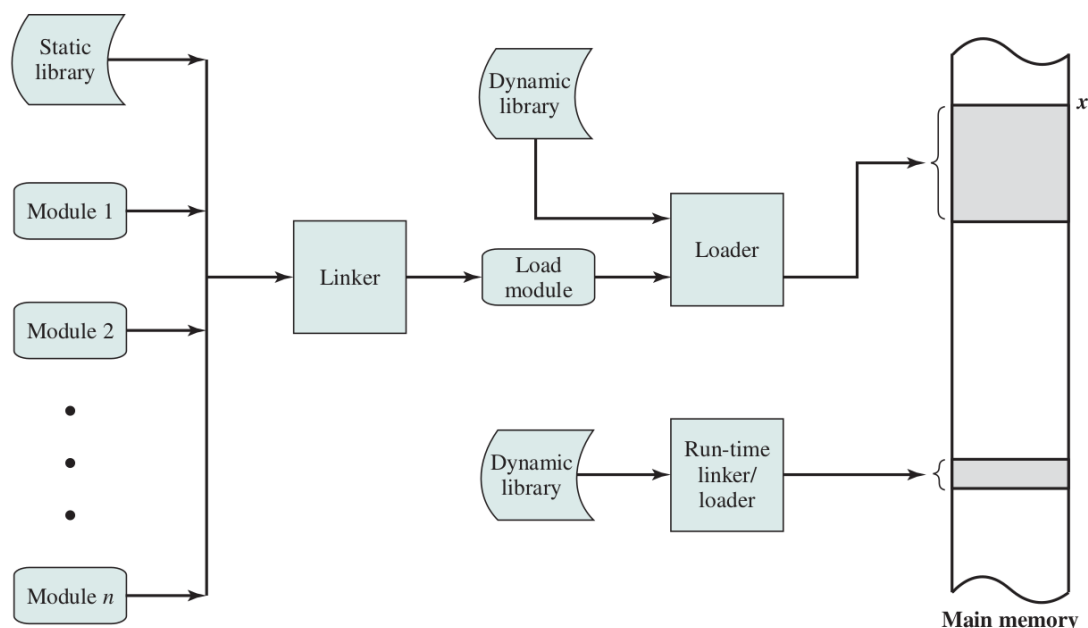
Memory management

Role

- The role of memory management in a multiprogramming environment focuses on allocating memory to processes and on controlling the transfer of data between main and secondary memory ([swapping area](#)), in order to
 - [Maintaining a register](#) of the parts of the main memory that are occupied and those that are free
 - [Reserving portions](#) of main memory for the processes that will need it, or [releasing](#) them when they are no longer needed
 - [Swapping out](#) all or part of the address space of a process when the main memory is too small to contain all the processes that coexist.
 - [Swapping in](#) all or part of the address space of a process when main memory becomes available

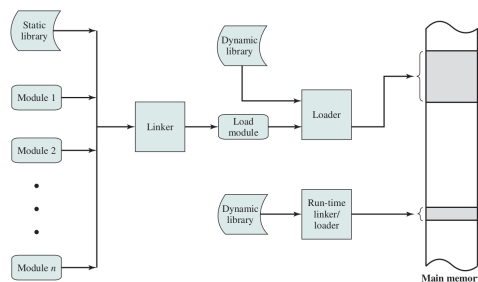
Address space

Linker and loader roles



Address space

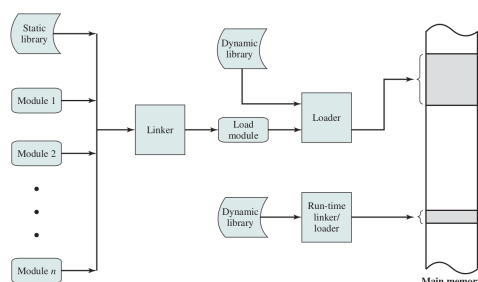
Linker and loader roles (2)



- The **object files**, resulting from the compilation process, are relocatable files
 - The addresses of the various instructions, constants and variables are calculated from the beginning of the module, by convention the address 0
- The role of the **linking process** is to bring the different object files together into a single file, the **executable file**, resolving among themselves the various external references
 - **Static libraries** are also included in the executable file
 - **Dynamic (shared) libraries** are not

Address space

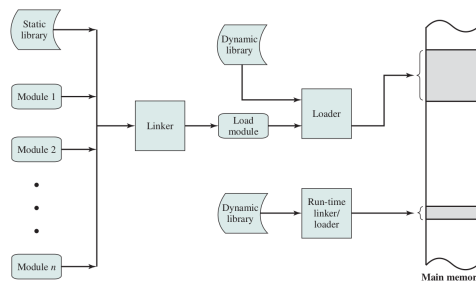
Linker and loader roles (3)



- The **loader** builds the binary image of the process **address space**, which will eventually be executed, combining the executable file and, if applicable, some **dynamic libraries**, resolving any remaining external references
- Dynamic libraries can also only be loaded at run time

Address space

Linker and loader roles (4)



- When the linkage is dynamic
 - Each reference in the code to a routine of a dynamic library is replaced by a **stub**
 - a small set of instructions that determines the location of a specific routine, if it is already resident in main memory, or promotes its load in memory, otherwise
 - When a stub is executed, the associated routine is identified and located in main memory, the stub then replaces the reference to its address in the process code with the address of the system routine and executes it
 - When that code zone is reached again, the system routine is now executed directly
- All processes that use the same dynamic library, execute the same copy of the code, thus minimizing the main memory occupation

Address space

Object and executable files

source file

```
#include <stdio.h>
#include <stdlib.h>

int main (void)
{
    printf ("hello, world!\n");
    exit (EXIT_SUCCESS);
}
```

object file

```
$ gcc -Wall -c hello.c

$ file hello.o
hello.o: ELF 64-bit LSB relocatable,
x86-64, version 1 (SYSV), not stripped
```

executable file

```
$ gcc -o hello hello.o

$ file hello
hello: ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically
linked, interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 3.2.0,
BuildID[sha1]=48ac0a8ba08d8df6d5e8a27e00b50248a3061876, not stripped
```

Address space

Object and executable files (2)

```
$ objdump -fstr hello.o
hello.o:      file format elf64-x86-64
architecture: i386:x86-64, flags 0x00000011:
HAS_RELOC, HAS_SYMS
start address 0x0000000000000000
SYMBOL TABLE:
0000000000000000 1      df *ABS*          0000000000000000 z.c
0000000000000000 1      d  .text          0000000000000000 .text
0000000000000000 1      d  .data          0000000000000000 .data
0000000000000000 1      d  .bss 0000000000000000 .bss
0000000000000000 1      d  .rodata        0000000000000000 .rodata
0000000000000000 1      d  .note.GNU-stack 0000000000000000 .note.GNU-stack
0000000000000000 1      d  .eh_frame        0000000000000000 .eh_frame
0000000000000000 1      d  .comment         0000000000000000 .comment
0000000000000000 g      F  .text          0000000000000000 1a main
0000000000000000          *UND*          0000000000000000 _GLOBAL_OFFSET_TABLE_
0000000000000000          *UND*          0000000000000000 puts
0000000000000000          *UND*          0000000000000000 exit

...
```

Address space

Object and executable files (3)

```
RELOCATION RECORDS FOR [.text]:
OFFSET          TYPE          VALUE
0000000000000007 R_X86_64_PC32      .rodata-0x0000000000000004
000000000000000c R_X86_64_PLT32     puts-0x0000000000000004
0000000000000016 R_X86_64_PLT32     exit-0x0000000000000004

RELOCATION RECORDS FOR [.eh_frame]:
OFFSET          TYPE          VALUE
0000000000000020 R_X86_64_PC32      .text

Contents of section .text:
0000 554889e5 488d3d00 000000e8 00000000  UH..H.=.....
0010 bf000000 00e80000 0000          .....
Contents of section .rodata:
0000 68656c6c 6f2c2077 6f726c64 2100      hello, world!.
Contents of section .comment:
0000 00474343 3a202855 62756e74 7520372e  .GCC: (Ubuntu 7.
0010 332e302d 32377562 756e7475 317e3138  3.0-27ubuntu1~18
0020 2e303429 20372e33 2e3000      .04) 7.3.0.
Contents of section .eh_frame:
0000 14000000 00000000 017a5200 01781001  ....zR..X..
0010 1b0c0708 90010000 1c000000 1c000000  ....
0020 00000000 1a000000 00410e10 8602430d  ....A....C.
0030 06000000 00000000          .....
```


Address space

Object and executable files (4)

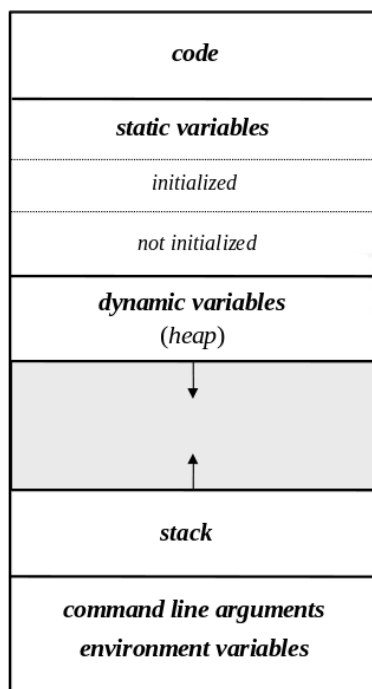
```
$ objdump -fTR hello
z:      file format elf64-x86-64
architecture: i386:x86-64, flags 0x00000150:
HAS_SYMS, DYNAMIC, D_PAGED
start address 0x0000000000000580

DYNAMIC SYMBOL TABLE:
0000000000000000 w D *UND*      0000000000000000
_ITM_deregisterTMCloneTable
0000000000000000 DF *UND*      0000000000000000 GLIBC_2.2.5 puts
0000000000000000 DF *UND*      0000000000000000 GLIBC_2.2.5 __libc_start_main
0000000000000000 w D *UND*      0000000000000000 __gmon_start__
0000000000000000 DF *UND*      0000000000000000 GLIBC_2.2.5 exit
0000000000000000 w D *UND*      0000000000000000
_ITM_registerTMCloneTable
0000000000000000 w DF *UND*      0000000000000000 GLIBC_2.2.5 __cxa_finalize

DYNAMIC RELOCATION RECORDS
OFFSET      TYPE      VALUE
0000000000200db0 R_X86_64_RELATIVE *ABS*+0x0000000000000680
0000000000200db8 R_X86_64_RELATIVE *ABS*+0x0000000000000640
0000000000201008 R_X86_64_RELATIVE *ABS*+0x0000000000201008
0000000000200fd8 R_X86_64_GLOB_DAT _ITM_deregisterTMCloneTable
0000000000200fe0 R_X86_64_GLOB_DAT __libc_start_main@GLIBC_2.2.5
0000000000200fe8 R_X86_64_GLOB_DAT __gmon_start__
0000000000200ff0 R_X86_64_GLOB_DAT _ITM_registerTMCloneTable
0000000000200ff8 R_X86_64_GLOB_DAT __cxa_finalize@GLIBC_2.2.5
0000000000200fc8 R_X86_64_JUMP_SLOT puts@GLIBC_2.2.5
0000000000200fd0 R_X86_64_JUMP_SLOT exit@GLIBC_2.2.5
```

Address space

Address space of a process



- **Code** and **static variables** regions have a fixed size, which is determined by the loader
- **Dynamic variables** and **stack** regions grow (in opposite directions) during the execution of the process
- It is a common practice to leave an unallocated memory area in the process address space between the dynamic definition region and the stack that can be used alternatively by any of them
- When this area is exhausted on the stack side, the execution of the process cannot continue, resulting in the occurrence of a fatal error: **stack overflow**

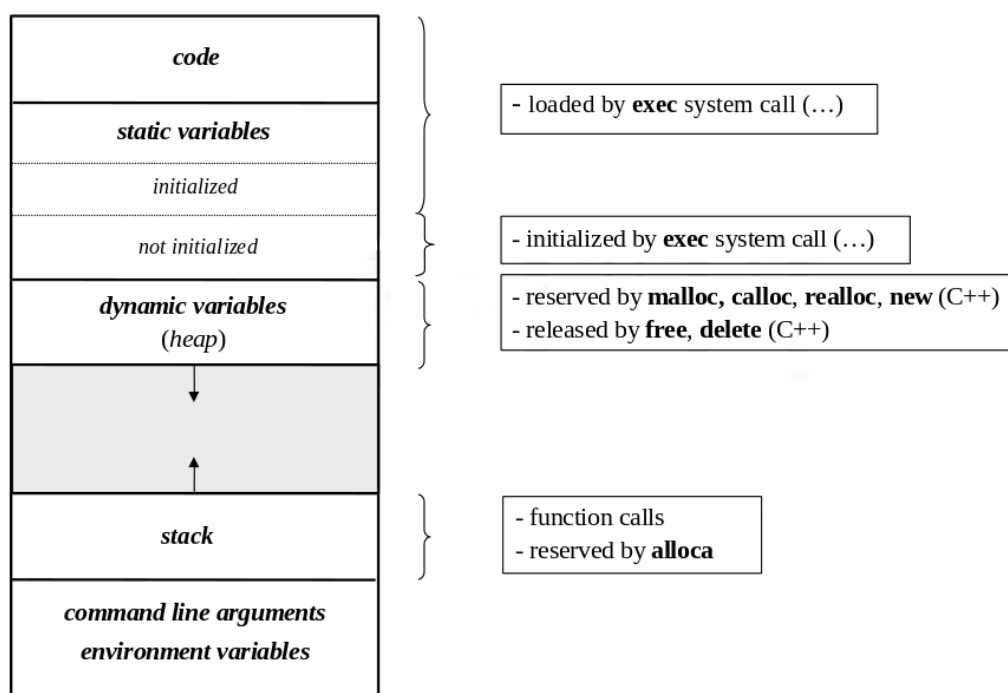
Address space

Address space of a process

- The binary image of the process address space represents a relocatable address space, the so-called **logical address space**
- The main memory region where it is loaded for execution, constitutes the **physical address space** of the process
- Separation between the logical and physical address spaces is a central concept to the memory management mechanisms in a multiprogrammed environment
- There are two issues that have to be solved
 - **dynamic mapping** – ability to convert a logical address to a physical address at runtime, so that the physical address space of a process can be placed in any region of main memory and be moved if necessary
 - **dynamic protection** – ability to prevent at runtime access to addresses located outside the process's own address space

Logical address space

Overview



Logical address space

Command line arguments and environment variables

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[], char *env[])
{
    /* printing command line arguments */
    printf("Command line arguments:\n");
    for (int i = 0; argv[i] != NULL; i++)
    {
        printf(" %s\n", argv[i]);
    }

    /* printing all environment variables */
    printf("\nEnvironment variables:\n");
    for (int i = 0; env[i] != NULL; i++)
    {
        printf(" %s\n", env[i]);
    }

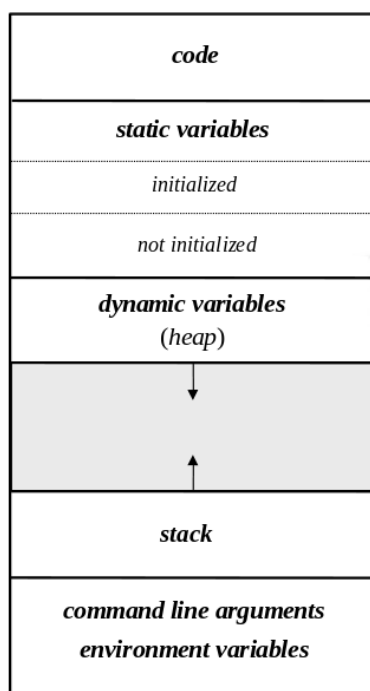
    /* printing a specific environment variable */
    printf("\nEnvironment variable:\n");
    printf(" env[\"HOME\"] = \"%s\"\n", getenv("HOME"));
    printf(" env[\"zzz\"] = \"%s\"\n", getenv("zzz"));

    return EXIT_SUCCESS;
}
```

- **argv** is an array of strings
- **argv[0]** is the program reference
- **env** is an array of strings, each representing a variable, in the form **name-value** pair
- **getenv** returns the value of a variable name

Logical address space

Logical addresses of variables



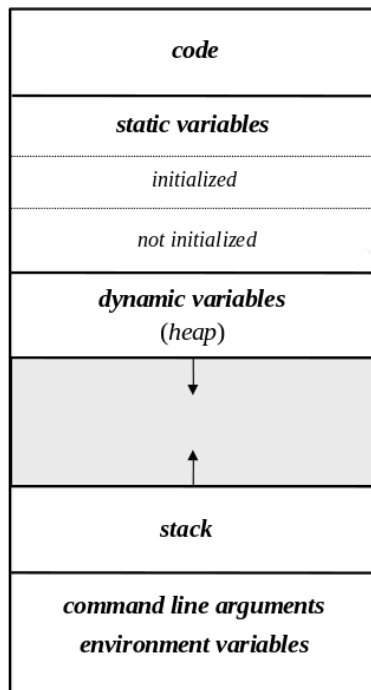
```
// n0 is defined in the environment
int n1 = 1;           // global, initialized
static int n2 = 2;    // static, file -scoped, initialized
int n3;               // global, not initialized
static int n4;        // static, file -scoped, not initialized
int n5;               // another global, not initialized
static int n6 = 6;    // another static, file -scoped, initialized

int main(int argc, char *argv[], char *env[])
{
    extern char** environ;
    static int n7;      // static, function-scoped, not initialized
    static int n8 = 8;  // static, function-scoped, initialized
    int *p9 = (int*) malloc(sizeof(int)); // heap-dynamic
    int *p10 = new int; // heap-dynamic
    int *p11 = (int*) alloca(sizeof(int)); // stack-dynamic
    int n12;           // local, not initialized
    int n13 = 13;       // local, initialized
    int n14;           // local, not initialized

    printf("\ngetenv(n0): %p\n", getenv("n0"));
    printf("\nargv: %p\nenviron: %p\nenv: %p\nmain: %p\n",
        argv, environ, env, main);
    printf("\n&argc: %p\n&argv: %p\n&env: %p\n",
        &argc, &argv, &env);
    printf("\nn1: %p\nn2: %p\nn3: %p\nn4: %p\nn5: %p\n"
        "n6: %p\nn7: %p\nn8: %p\nn9: %p\nn10: %p\n"
        "p11: %p\nn12: %p\nn13: %p\nn14: %p\n",
        &n1, &n2, &n3, &n4, &n5, &n6, &n7, &n8,
        p9, p10, p11, &n12, &n13, &n14);
}
```

Logical address space

Logical address space after a `fork`



```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <wait.h>

int n01 = 1;

int main(int argc, char *argv[], char *env[])
{
    int pid = fork();
    if (pid != 0)
    {
        fprintf(stderr, "%5d: n01 = %5d (%p)\n",
                pid, n01, &n01);
        wait(NULL);
        fprintf(stderr, "%5d: n01 = %5d (%p)\n",
                pid, n01, &n01);
    }
    else
    {
        fprintf(stderr, "%5d: n01 = %5d (%p)\n",
                pid, n01, &n01);
        n01 = 1111;
        fprintf(stderr, "%5d: n01 = %5d (%p)\n",
                pid, n01, &n01);
    }
    return 0;
}
```

Logical address space

Logical addresses between threads

```
void *threadChild (void *par)
{
    printf ("I'm the child thread! (PID: %d; TID: %d)\n", getpid(), gettid());

    int n1 = 0;
    static int n2 = 0;
    printf("[%u] &n1: %p; &n2: %p\n", gettid(), &n1, &n2);

    return NULL;
}

int main (int argc, char *argv[])
{
    printf ("I'm the main thread! (PID: %d)\n", getpid());

    threadChild(NULL);
    threadChild(NULL);

    pthread_t thr[2];
    for (int i = 0; i < 2; i++) {
        if (pthread_create (&thr[i], NULL, threadChild, NULL) != 0) {
            perror ("Fail launching thread");
            return EXIT_FAILURE;
        }
    }

    for (int i = 0; i < 2; i++) {
        if (pthread_join (thr[i], NULL) != 0) {
            perror ("Fail joining child thread");
            return EXIT_FAILURE;
        }
    }

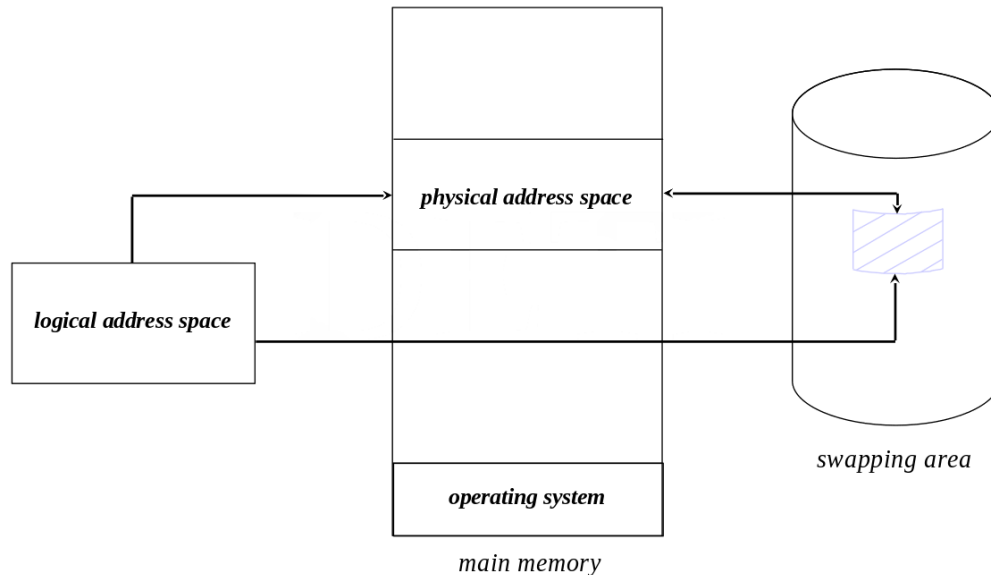
    threadChild(NULL);

    return EXIT_SUCCESS;
}
```

Contiguous memory allocation

Logical and physical address spaces

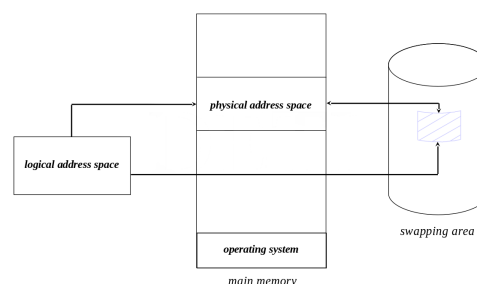
- In contiguous memory allocation, there is a **one-to-one correspondence** between the **logical address space** of a process and its **physical address space**



Contiguous memory allocation

Logical and physical address spaces

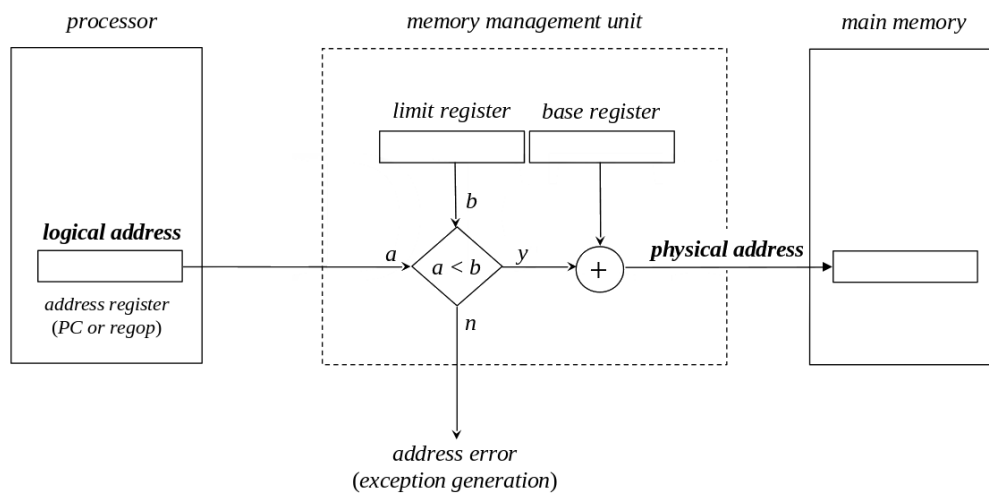
- Consequences:
 - Limitation of the address space of a process** – in no case can memory management support automatic mechanisms that allow the address space of a process to be larger than the size of the main memory available
 - The use of overlays can allow to overcome that
 - Contiguity of the physical address space** – although it is not a strictly necessary condition, it is naturally simpler and more efficient to assume that the process address space is contiguous
 - Swapping area as an extension of the main memory** – it serves to storage the address space of processes that cannot be resident into main memory due to lack of space



Contiguous memory allocation

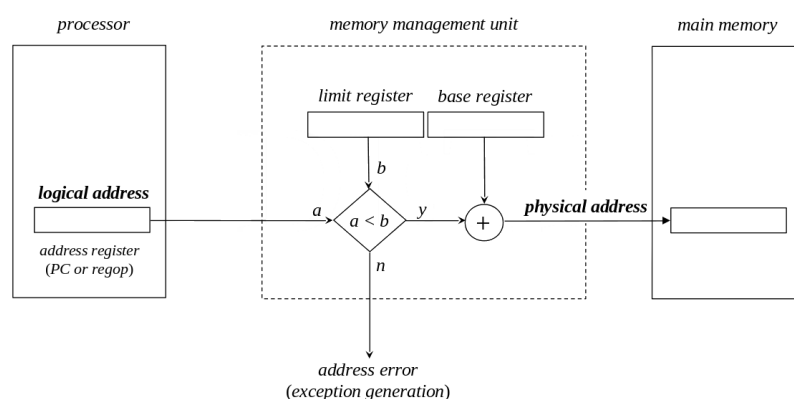
Logical address to physical address translation

- How are **dynamic mapping** and **dynamic protection** accomplished?
 - A piece of hardware (the MMU) comes into play



Contiguous memory allocation

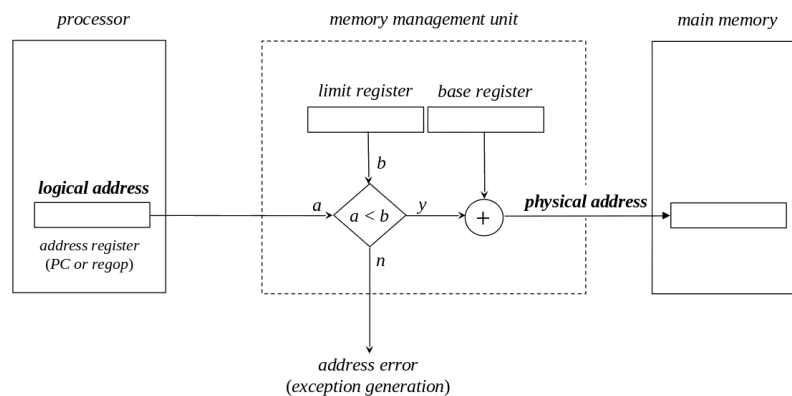
Logical address to physical address translation (2)



- The **limit register** must contain the size in bytes of the logical address space
- The **base register** must contain the address of the beginning of the main memory region where the logical address space of the process is placed
- On context switching, the **dispatch** operation loads the base and limit registers with the values present in the corresponding fields of the process control table entry associated with the process that is being scheduled for execution

Contiguous memory allocation

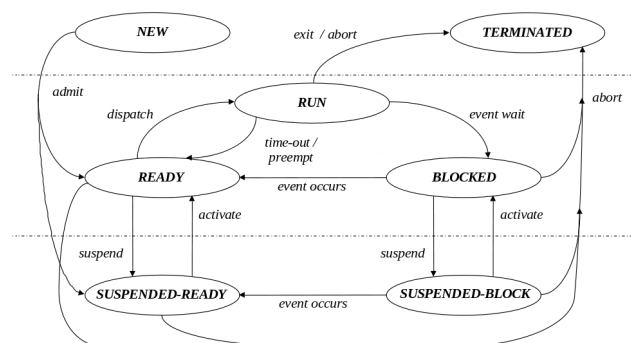
Logical address to physical address translation (2)



- Whenever there is a reference to memory
 - the logical address is first compared to the value of the limit register
 - if it is greater than or equal to, it is an invalid reference, a null memory access (dummy cycle) is set in motion and an exception is generated due to address error
 - otherwise, it is a valid reference (it occurs within the process address space), the logical address is added to the value of the base register to produce the physical address

Contiguous memory allocation

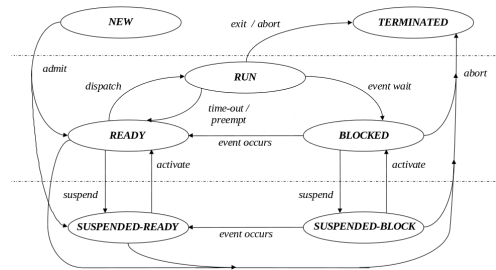
Long-term scheduling



- When a process is created, the data structures to manage it is initialized
 - Its logical address space is constructed, and the value of the limit register is computed and saved in the corresponding field of the process control table (PCT)
- If there is space in main memory, its address space is loaded there, the base register field is updated with the initial address of the assigned region and the process is placed in the READY queue
- Otherwise, its address space is temporarily stored in the swapping area and the process is placed in the SUSPENDED-READY queue

Contiguous memory allocation

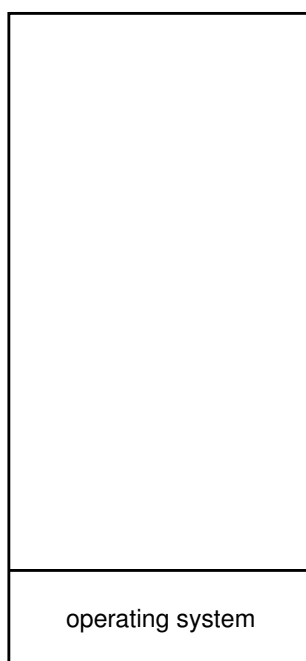
Medium-term scheduling



- If memory is required for another process, a BLOCKED (or even READY) process may be **swapped out**, freeing the physical memory it is using,
 - In such a case, its base register field in the PCT becomes undefined
- If memory becomes available, a SUSPENDED-READY (or even SUSPENDED-BLOCKED) process may be **swapped in**,
 - Its base register field in the PCT is updated with its new physical location
 - A SUSPENDED-BLOCK process is only selected if no SUSPENDED-READY one exists
- When a process terminates, it is **swapped out** (if not already there), waiting for the end of operations

Memory partitioning

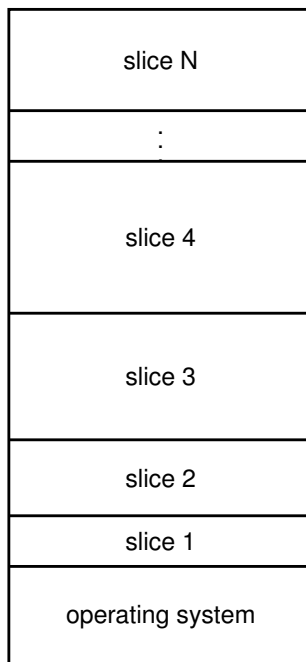
How to do it?



- After reserving some amount to the operating system, how to partition the main memory to accommodate the different processes?
 - Fixed-size partitioning
 - into slices of equal size
 - into slices of different size
 - Dynamic partitioning
 - being done as being requested

Memory partitioning

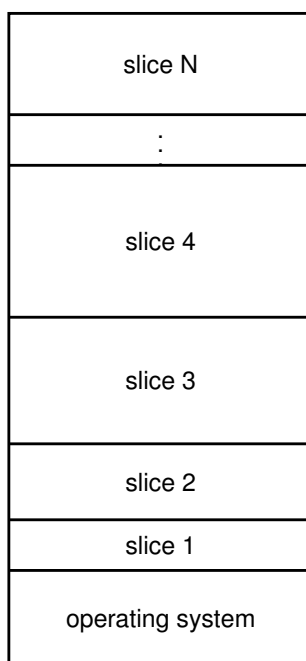
Fixed-size partitioning



- Main memory can be divided into a number of static slices at system generation time
 - not necessarily all the same size
- The logical address space of a process may be loaded into a slice of equal or greater size
 - thus, the largest slice determines the size of the largest allowable process
- Some features:
 - Simple to implement
 - Efficient – little operating system overhead
 - Fixed number of allowable processes
 - Inefficient use of memory due to internal fragmentation – the part of a slice not used by a process is wasted

Memory partitioning

Fixed-size partitioning (2)

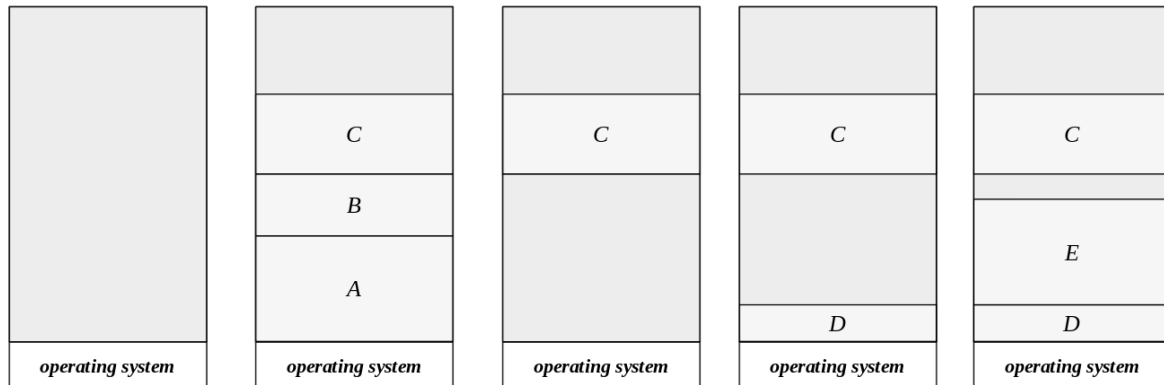


- If a slice becomes available, which of the SUSPENDED processes should be placed there?
- Two different scheduling policies are here considered
 - **Valuing fairness** – the first process in the queue of SUSPENDED-READY processes whose address space fits in the slice is chosen
 - **Valuing the occupation of main memory** – the first process in the queue of SUSPENDED-READY processes with the largest address space that fits in the slice is chosen
 - to avoid starvation an aging mechanism can be used

Memory partitioning

Dynamic partitioning

- In dynamic partitioning, at start, all the available part of the memory constitutes a single block and then
 - reserve a region of sufficient size to load the address space of the processes that arises
 - release that region when it is no longer needed



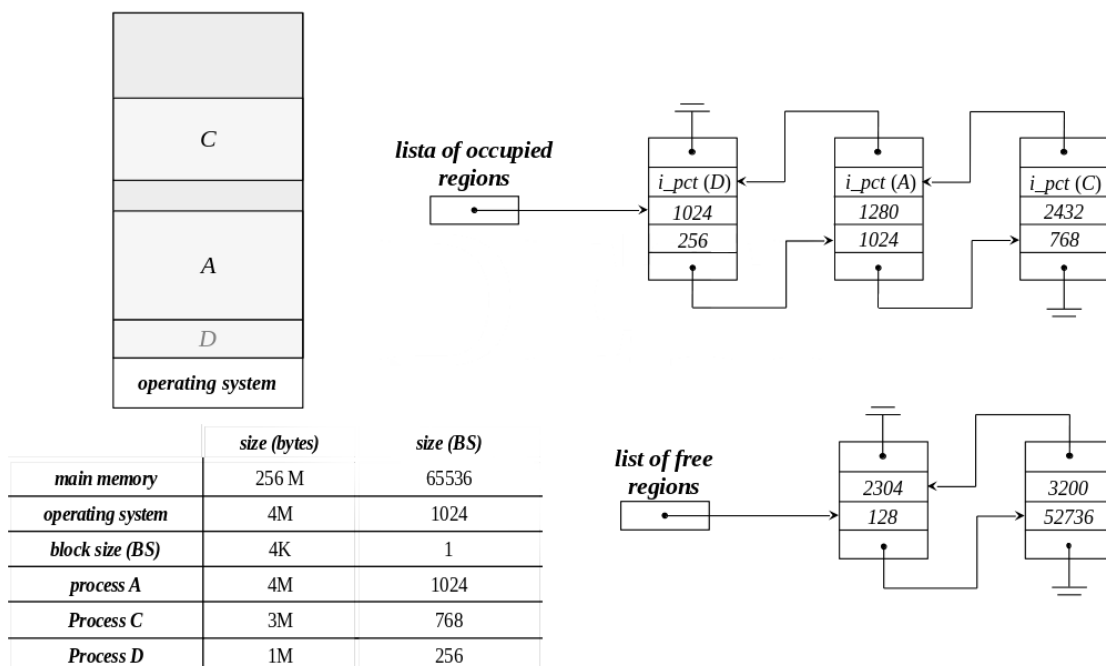
Memory partitioning

Dynamic partitioning (2)

- As the memory is dynamically reserved and released, the operating system has to keep an updated record of occupied and free regions
- One way to do this is by building two (bi)linked lists
 - [list of occupied regions](#) – locates the regions that have been reserved for storage of the address spaces of processes resident in main memory
 - [list of free regions](#) – locates the regions still available
- Memory is not allocated in byte boundaries, because
 - useless, very small free regions may appear...
 - that will be included in the list of free regions...
 - making subsequent searches more complex
- Thus, the main memory is typically divided into blocks of fixed size and allocation is made in units of these blocks

Memory partitioning

Dynamic partitioning (3)



Memory partitioning

Dynamic partitioning (4)

- **Valuing fairness** is the scheduling discipline generally adopted, being chosen the first process in the queue of SUSPENDED-READY processes whose address space can be placed in main memory
- Dynamic partitioning can produce **external fragmentation**
 - Free space is splitted in a large number of (possible) small free regions
 - Situations can be reached where, although there is enough free memory, it is not continuous and the storage of the address space of a new or suspended process is no longer possible
- The solution is **garbage collection** – compact the free space, grouping all the free regions into a single one
 - This operation requires stopping all processing and, if the memory is large, can have a very long execution time

Memory partitioning

Dynamic partitioning (5)

- In case there are several free regions available, which one to use to allocate the address space of a process?
- Possible policies:
 - **first fit** – the list of free regions is searched from the beginning until the first region with sufficient size is found
 - **next fit** – is a variant of the first fit which consists of starting the search from the stop point in the previous search
 - **best fit** – the list of free regions is fully searched, choosing the smallest region with sufficient size for the process
 - **worst fit** – the list of free regions is fully searched, choosing the largest existing region
 - **buddy system** – which uses a binary tree to represent used or unused memory blocks and splits memory into halves to try to give a best fit
- Which one is the best?
 - in terms of fragmentation
 - in terms of efficiency of allocation
 - in terms of efficiency of release

Memory partitioning

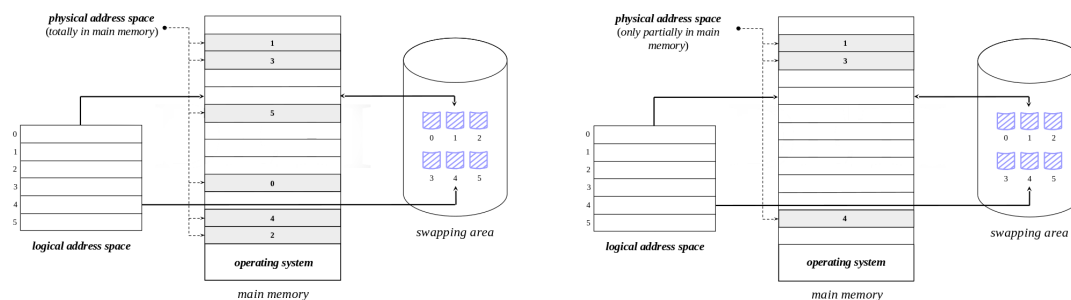
Dynamic partitioning (6)

- Advantages
 - **general** – the scope of application is independent of the type of processes that will be executed
 - **low complexity implementation** – no special hardware required and data structures are reduced to two (bi)linked lists
- Disadvantages
 - **external fragmentation** – the fraction of the main memory that ends up being wasted, given the small size of the regions in which it is divided, can reach in some cases about a third of the total
 - **inefficient** – it is not possible to build algorithms that are simultaneously very efficient in allocating and freeing space

Virtual memory system

Mapping of the logical address space

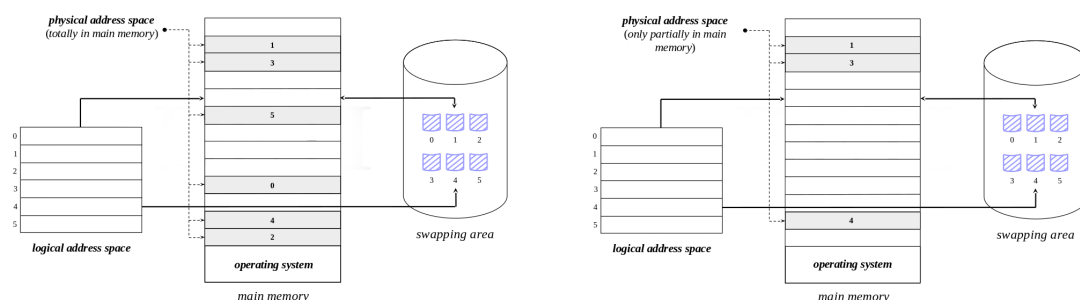
- In a virtual memory system, the **logical address space** of a process and its **physical address space** are **totally dissociated**
- The **logical address space** is sliced in different blocks
- The different blocks are allocated independently of each other
 - So they can spread along the physical address space
- A process can be only partially resident in main memory
 - So some of its blocks may be only in the swapping area



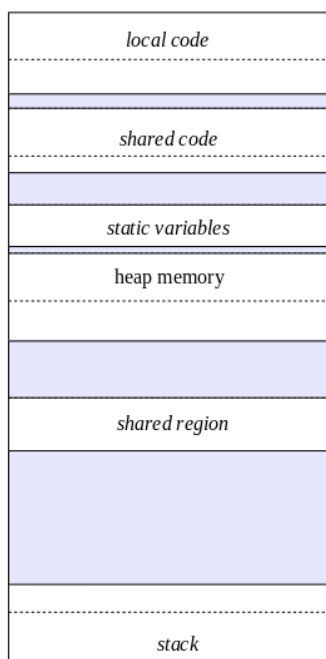
Virtual memory system

Some features

- **Non-contiguity of the physical address space** – the address spaces of the processes, divided into blocks of fixed or variable size, are dispersed throughout the memory, trying to guarantee a more efficient occupation of the available space
- **No limitation of the address space of a process** – methodologies allowing the execution of processes whose address spaces are greater than the size of the available main memory can be established
- **Swapping area as an extension of the main memory** – its role is to maintain an updated image of the address spaces of the processes that currently coexist, namely their variable part (static and dynamic definition areas and stack)

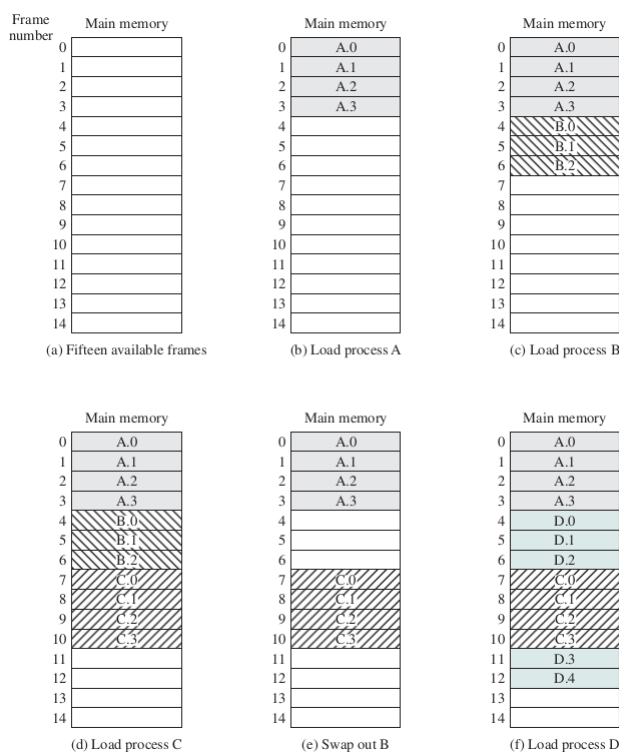


Paging Introduction



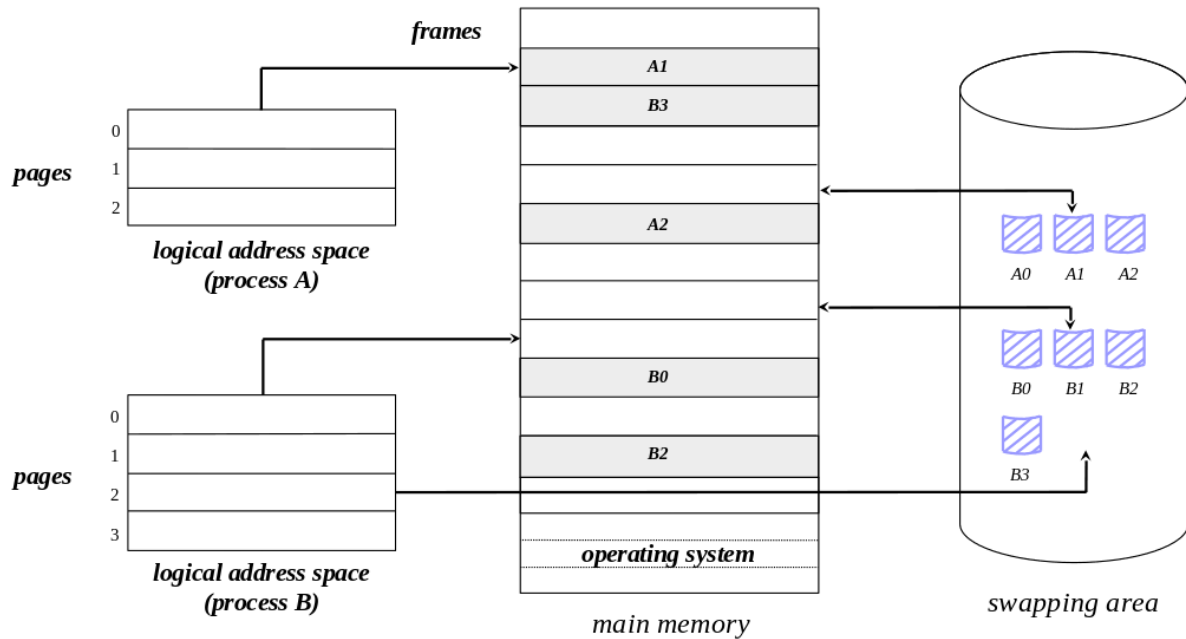
- Memory is divided into equal fixed-size chunks, called **frames**
 - a power of 2 is used for the size, typically 4 or 8 KB
- The logical address space of a process is divided into fixed-size blocks, of the same size, called **pages**
- While dividing the address space into pages, the linker usually starts a new page when a new segment starts
- In a logical address:
 - the most significant bits represent the **page number**
 - the least significant bits represent an **offset** within the page

Paging Illustration example



Paging

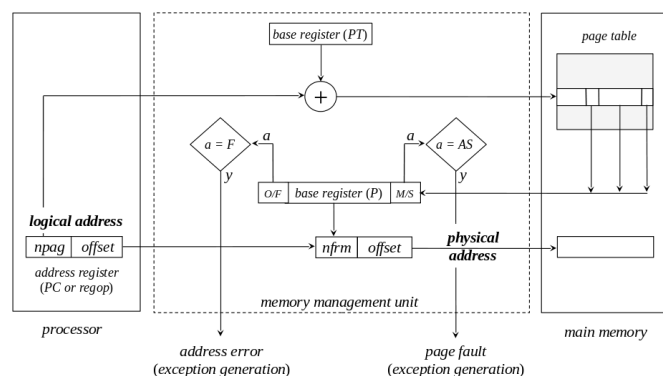
Illustration example



Paging

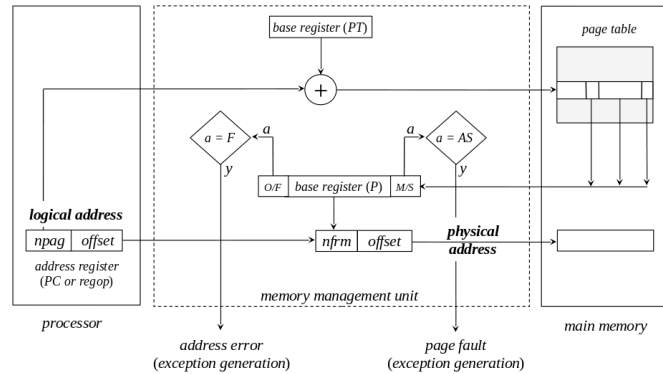
Logical address to physical address translation

- How are **dynamic mapping** and **dynamic protection** accomplished?
- A **logical address** is composed of two parts:
 - **npag** – that identifies a specific logical block (page)
 - **offset** – that identifies a position within the page, as an offset from its beginning
- A **page table**, stored in memory, maps every page to its physical counterpart (frame)
- The **MMU** must deal with this structure



Paging

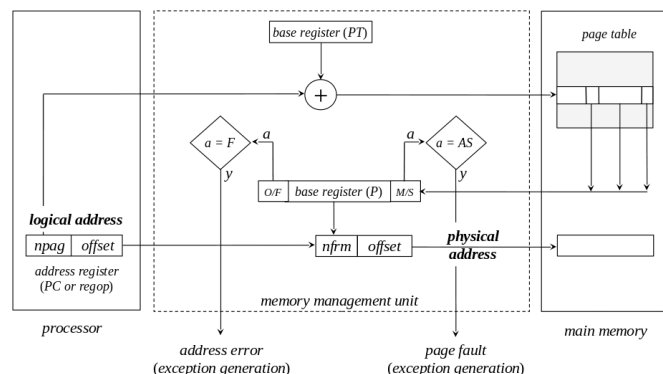
Logical address to physical address translation



- A **base register** is required to store the start address of the page table
- The logical address space of the process is structured in order to map the whole, or at least a fraction, of the address space provided by the processor (in any case, always greater than or equal to the size of the existing main memory), so all page tables have the same size, eliminating the need of a **limit register** associated with the size of the page table
- The gap between the heap memory and the stack can be maximized

Paging

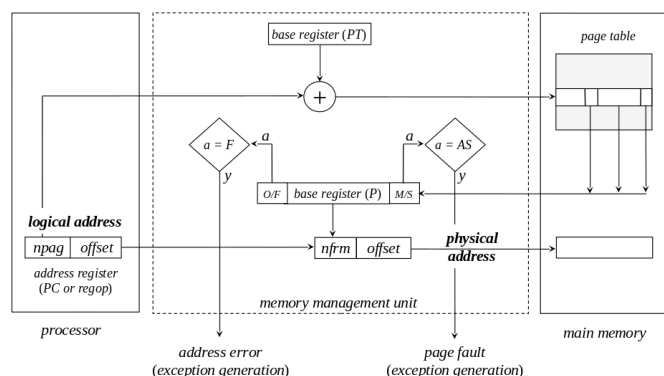
Logical address to physical address translation



- Another **base register** is required to store the start address of the current page
- Since all pages have the same size, there is no need for a **limit register** associated to the page.

Paging

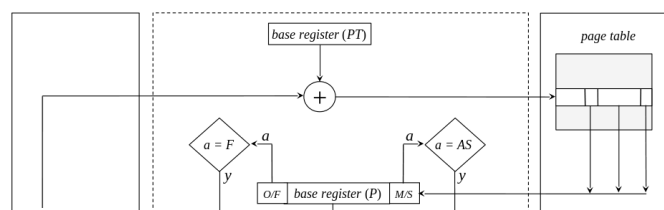
Logical address to physical address translation



- On context switching, the **dispatch** operation loads the **page table base register** with the start address of the page table of the process scheduled for execution
- Every time the running process access an address out of the current page, the **page base register** must be reloaded
 - The new value of this register must be obtained from the page table, unless it is already cached by the TLB

Paging

Page table entries



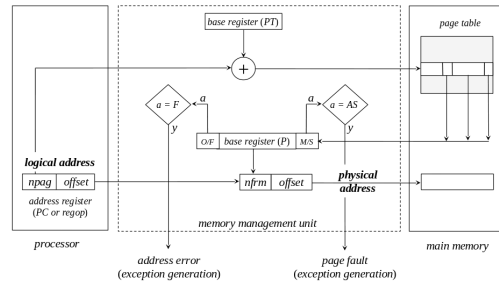
- Page table contains one entry per page
- Entry definition:

O/F	M/S	ref	mod	perm	frame number	block number in swap area
-----	-----	-----	-----	------	--------------	---------------------------

- O/F** – flag indicating if page has been already assigned to process
- M/S** – flag indicating if page is in memory
- ref** – flag indicating if page has been referenced
- mod** – flag indicating if page has been modified
- perm** – permissions
- frame number** – frame where page is, if in memory (base register of page)
- block number in swap area** – block where page is, in swapping area

Paging

Logical address to physical address translation



- In a first step, the MMU must add the **page table base register** with the **page number**, obtaining a pointer to the **page table entry**, loading its contents to the MMU
- Then, if flag **O/F** is **O** (the page being accessed is valid), the operation may proceed
 - if not (the page being accessed was not assigned to the process), a null memory access is set in motion and an **exception** is generated due to **address error**
- Then, if flag **M/S** is **M** (the page being accessed is in memory), the operation may proceed
 - if not (the page being referenced is swapped out), a null memory access (dummy cycle) is set in motion and an **exception** is generated due to **page fault**
- Finally, the **page base register** (containing the frame number) is concatenated with the **offset** to obtain the physical address of the memory position being accessed

Paging

Role of the TLB

- The need for this double access to memory can be minimized by taking advantage of the **principle of locality of reference**
- As the accesses will tend to be concentrated in a well-defined set of blocks during extended process execution time intervals, the memory management unit (MMU) usually keeps the content of the page table entries stored in an internal associative memory, called the **translation lookaside buffer (TLB)**
- Thus, the access to a page can be a
 - **hit** – when the entry is stored in the TLB, in which case the access is internal to the MMU
 - **miss** – when the entry is not stored in the TLB, in which case there is access to the main memory
- The average access time to an instruction or operand tends to approximate the lowest value
 - an access to the TLB plus an access to the main memory

Paging

Paging with TLB

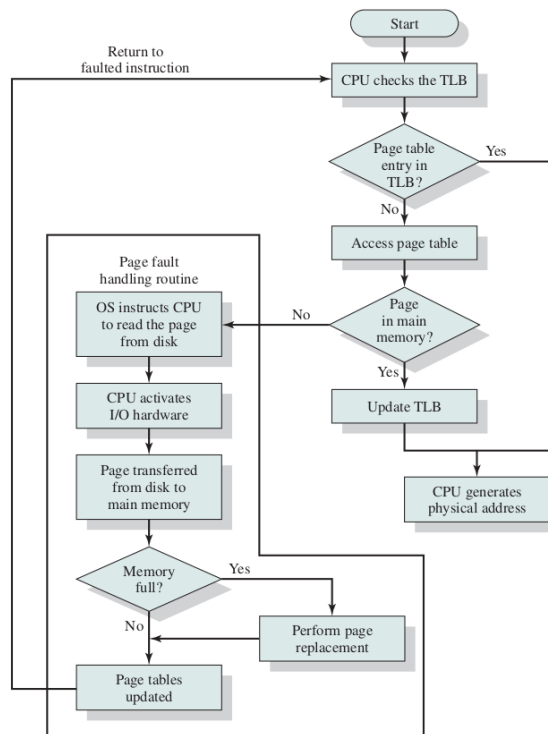


Figure 8.8 Operation of Paging and Translation Lookaside Buffer (TLB)

Operating Systems: Internals and Design Principles, William Stallings

Paging

Analysis

Advantages:

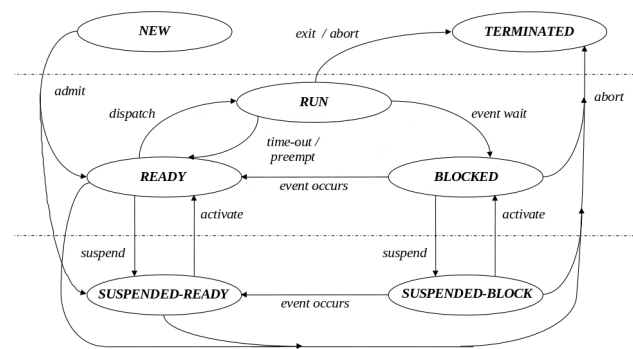
- **general** – the scope of application is independent of the type of processes that will be executed (number and size of their address spaces)
- **good usage of main memory** – does not lead to external fragmentation and internal fragmentation is practically negligible
- **does not have special hardware requirements** – the memory management units in today's general-purpose processors implements it

Disadvantages:

- **longer memory access** – double access to memory, because of a prior access to the page table
 - Existence of a TLB (translation lookaside buffer) minimizes the impact
- **very demanding operability** – requires the existence of a set of support operations, that are complex and have to be carefully designed to not compromise efficiency

Paging virtual memory system

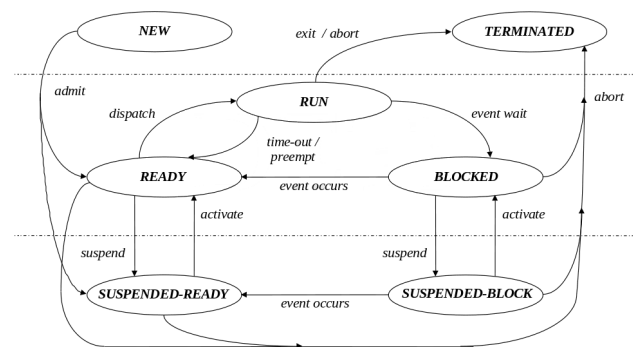
Long-term scheduling



- When a process is created, the data structures to manage it is initialized
 - Its logical address space is constructed, and its page table is organized
 - Some pages can be shared with other processes
- If there is space in main memory, at least its page table, first page of code and the page of its stack are loaded there, the corresponding entries in the page table are updated and the process is placed in the READY queue
- Otherwise, the process is placed in the SUSPENDED-READY queue

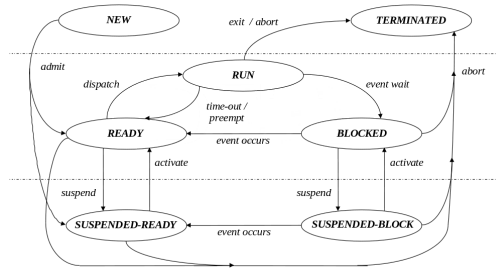
Paging virtual memory system

Short-term scheduling



- During its execution, on **page fault**, a process is placed in the BLOCKED state, while the **faulty page is swapped in**
- When the page is in memory, the process is placed in the READY state

Medium-term scheduling



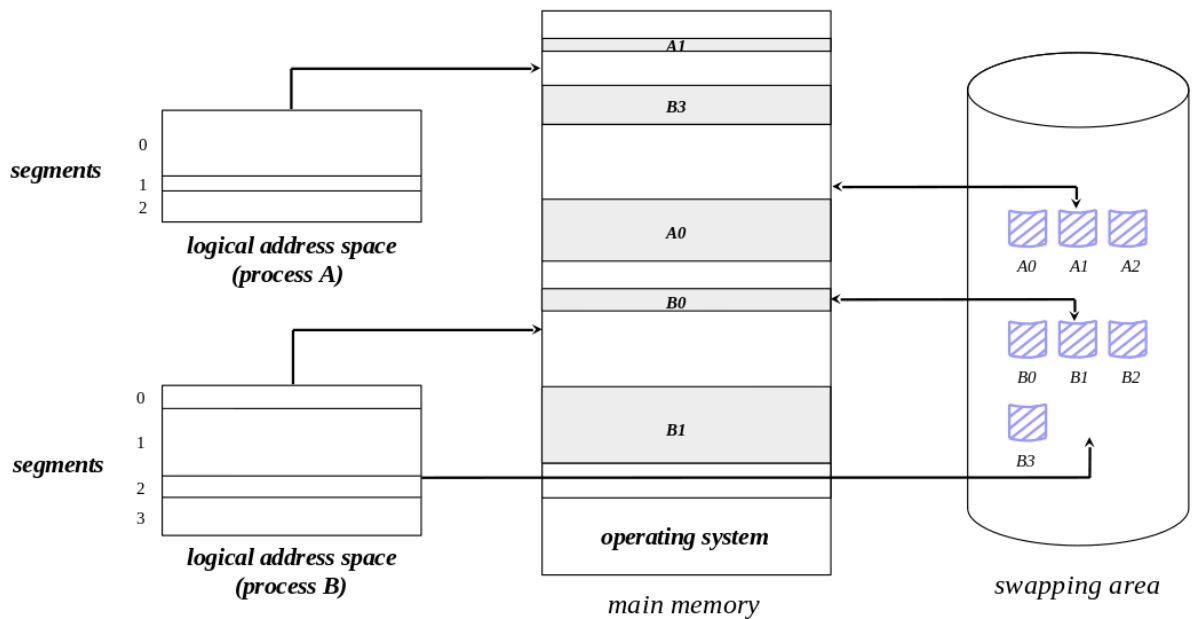
- While READY or BLOCKED, all pages of a process may be **swapped out**, if memory space is required
- While SUSPENDED-READY (or SUSPENDED-BLOCKED), if memory space becomes available, the page table and a selection of pages of a process may be **swapped in**, and the process transitions to READY or BLOCKED
 - the corresponding entries of the page table are updated
 - A SUSPENDED-BLOCK process is only selected if no SUSPENDED-READY one exists
- When a process terminates, it may be **swapped out** (if not already there), waiting for the end of operations

Introduction

- Typically, the logical address space of a process is composed of different type of segments:
 - **code** – one segment per code module
 - **static variables** – one segment per module containing static variables
 - **heap memory** – one segment
 - **shared memory** – one segment per shared region
 - **stack** – one segment
 - Different segments may have different sizes
- In a **segmentation architecture**, the segments of a process are manipulated separately
 - **Dynamic partitioning** may be used to allocate each segment
 - As a consequence, a process may not be contiguous in memory
 - Even, some segments may not be in main memory

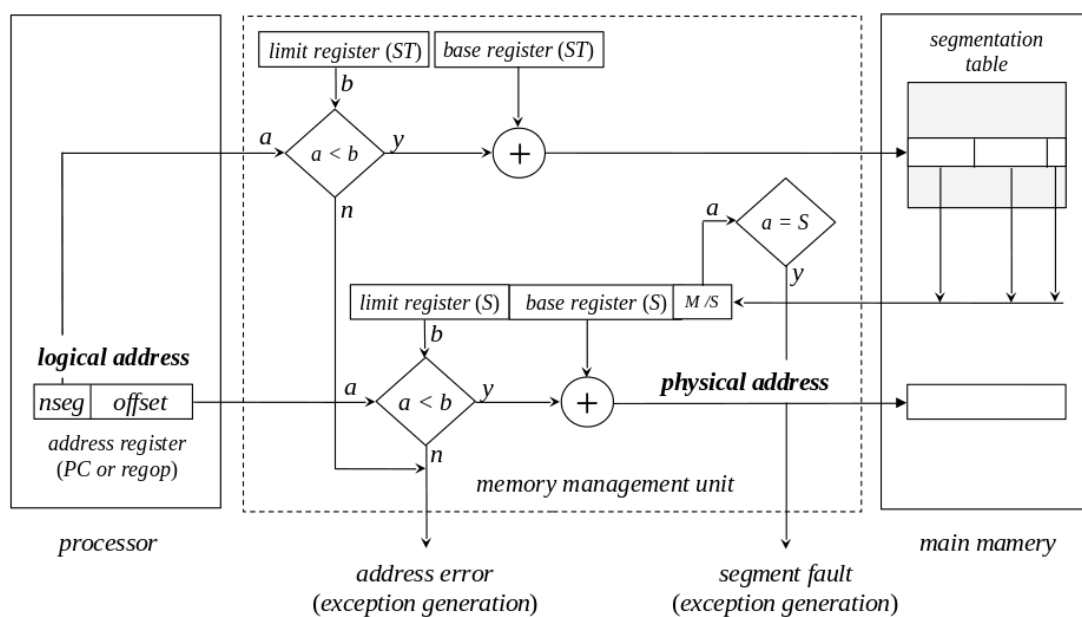
Segmentation

Illustration example



Segmentation

Memory management unit (MMU)



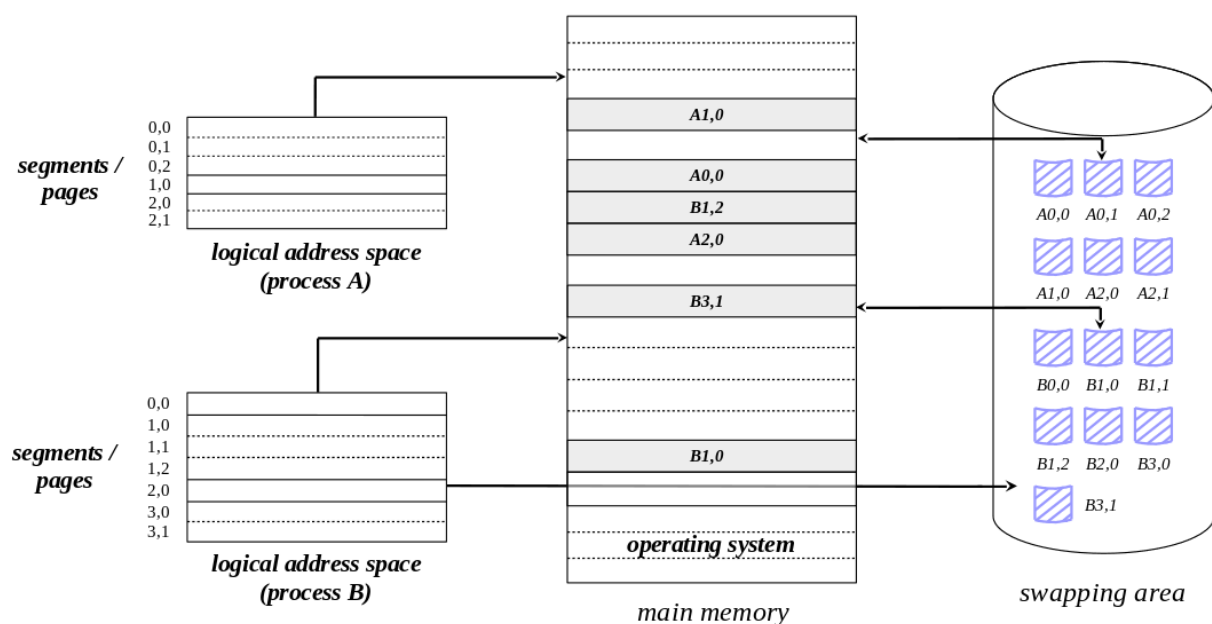
Combining segmentation and paging

Introduction

- Segmentation, taken alone, can have some drawbacks:
 - It may result in external fragmentation
 - A growing segment can impose a change in its location
- Merging segmentation and paging can solve these issues
 - First, the logical address space of a process is partitioned into segments
 - Then, each segment is divided into pages
- However, this introduces a growing complexity

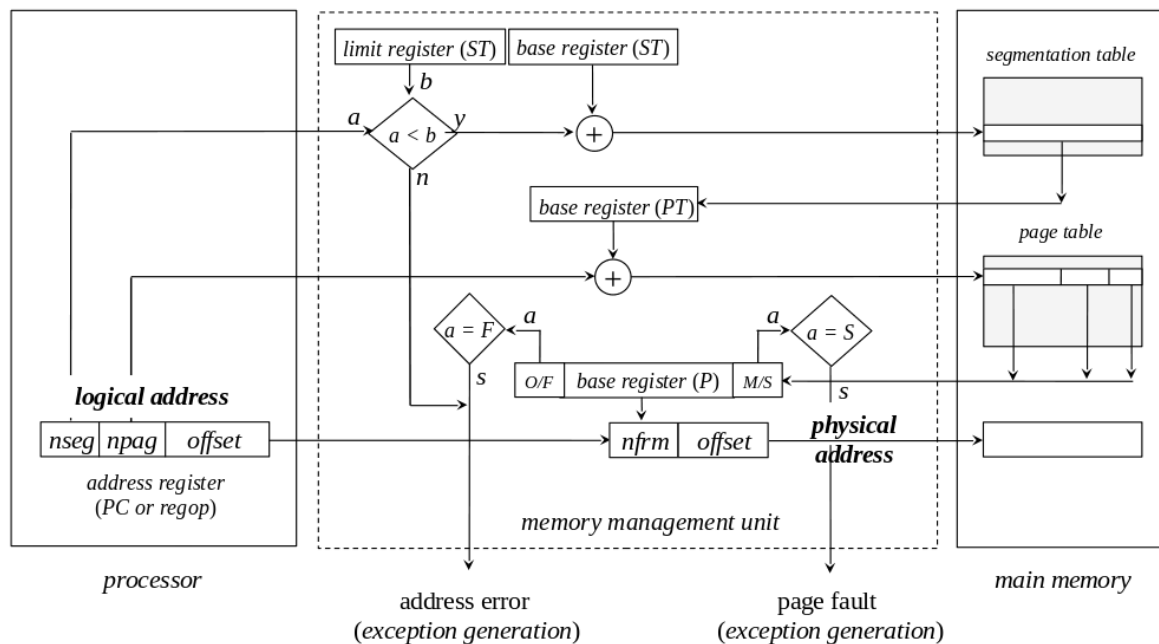
Combining segmentation and paging

Illustration example



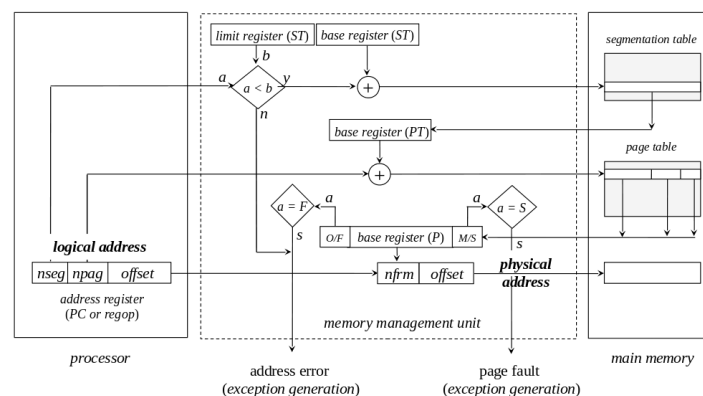
Combining segmentation and paging

Memory management unit (MMU)



Combining segmentation and paging

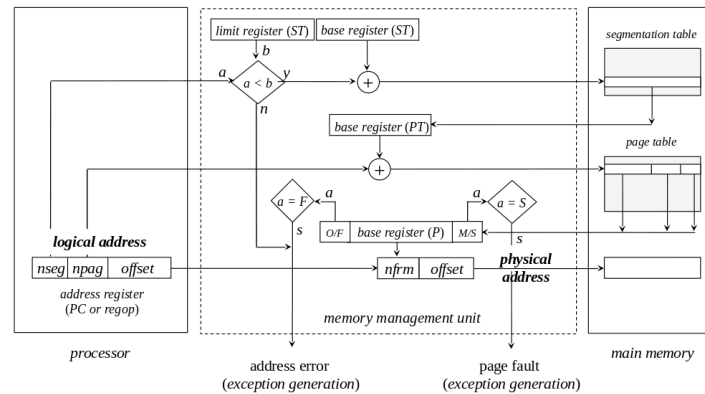
Logical address to physical address translation



- The MMU must contain 3 base registers and 1 limit register
 - 1 base register for the segmentation table
 - 1 limit register for the segmentation table
 - 1 base register for the page table
 - 1 base register for the memory frame

Combining segmentation and paging

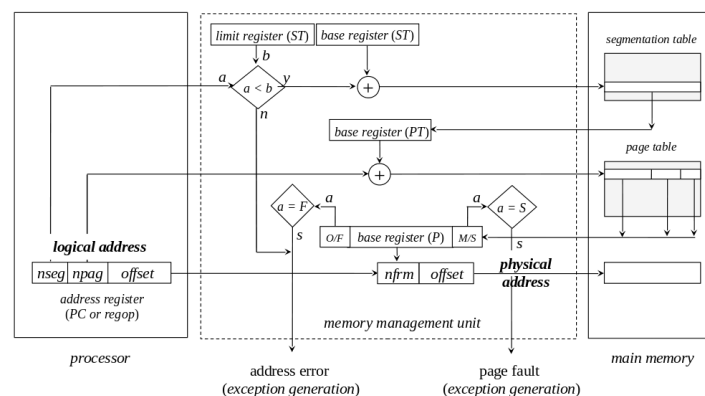
Logical address to physical address translation (2)



- An access to memory unfolds into 3 steps:
 - Access to the segmentation table
 - Access to the page table
 - Access to the physical address

Combining segmentation and paging

Logical address to physical address translation (3)



- Entry of the segmentation table

perm	memory address of the page table
------	----------------------------------

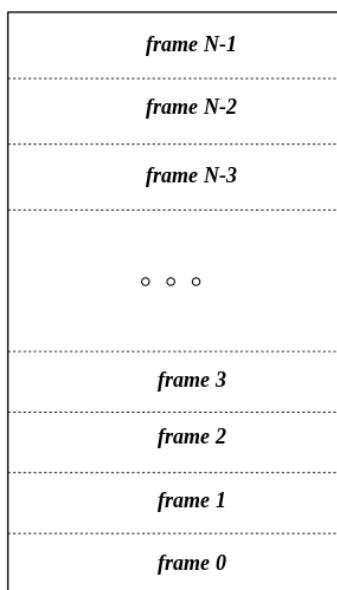
- Entry of the page table

O/F	M/S	ref	mod	frame number	block number in swap area
-----	-----	-----	-----	--------------	---------------------------

- The `perm` field is now associated to the segment

Page replacement

Introduction

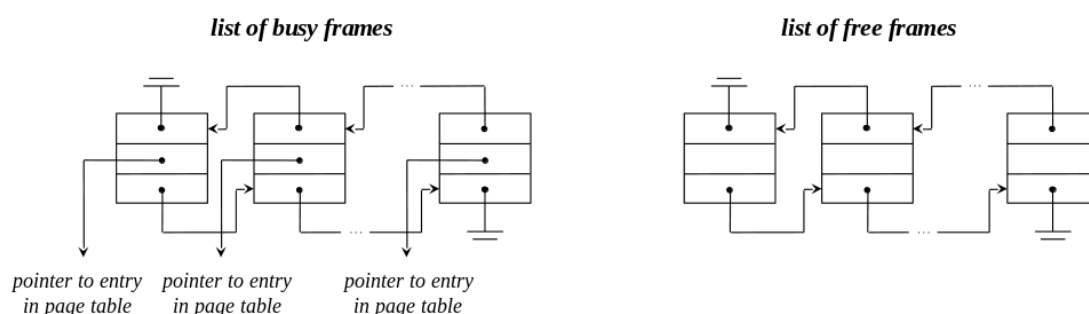


- In a paging (or combination of segmentation and paging) architecture, memory is partitioned into frames, each the same size as a page
- A frame may be either **free** or **occupied** (containing a page)
- A page in memory may be:
 - **locked** – if it can not be removed from memory (kernel, buffer cache, memory-mapped file)
 - **unlocked** – if it can be removed from memory
- If no free frame is available, an occupied one may need to be released
 - This is the purpose of **page replacement**
- Page replacement only applies to unlocked pages

Page replacement

Lists of free and occupied frames

- Free frames are organized in a list – **list of free frames**
- Occupied frames, associated to unlocked pages, are also organized in a list – **list of occupied frames**
- How the list of occupied frames is organized depends on the **page replacement policy**



Page replacement

Action on page fault

- On **page fault**, if the list of free frames is empty, an occupied frame must be selected for replacement
- Alternatively, the system can promote page replacement as to maintain the list of free frames always with some elements
 - This allows to load the faulty page and free a busy frame at the same time
- The question is: which frame should be selected for replacement?
- An **optimal policy** selects for replacement that page for which the time to the next reference is the longest
 - Unless we have a Crystal-Ball, it is **impossible to implement**
 - But, useful as benchmark
- Covered algorithms:
 - **Least Recently Used (LRU)**
 - **Not Recently Used (NRU)**
 - **First In First Out (FIFO)**
 - **Second chance**
 - **Clock**

Page replacement policies

LRU algorithm

- The **Least Recently Used** policy selects for replacement the frame that has not been referenced the longest
 - Based on the principle of locality of reference, if a frame is not referenced for a long time, it is likely that it will not be referenced in the near future
- Each frame must be labelled with the time of the last reference
 - Additional specific hardware may be required
- On page replacement, the list of occupied frames must be traversed to find out the one with the oldest last access time
- High cost of implementation and not very efficient

Page replacement policies

NRU algorithm

- The **Not Recently Used** policy selects for replacement a frame based on classes
- Bits **Ref** and **Mod**, fields of the entry of the page table, and typically processed by conventional MMU, are used to define classes of frames

class	Ref	Mod
0	0	0
1	0	1
2	1	0
3	1	1

- On page replacement, the algorithm selects at random a frame from the lowest non-empty class
- Periodically, the system traverse the list of occupied frames and put **Ref** at zero

Page replacement policies

FIFO algorithm

- The **FIFO** policy selects for replacement based on the length of stay in memory
 - Based on the assumption that the longer a page resides in memory, the less likely it is to be referenced in the future
- The list of occupied frames is considered to be organized in a FIFO that reflects the loading order of the corresponding pages in main memory
- On page replacement, the frame with the oldest page is selected
- The assumption in itself is extremely fallible
 - Consider for instance system shared libraries
 - But can be interesting with a refinement

Page replacement policies

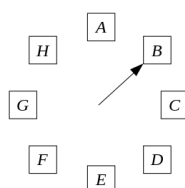
Second chance algorithm

- The **second chance** policy is an improvement of the FIFO algorithm, giving a page a second chance before it is replaced
- On page replacement:
 - The frame with the oldest page is selected as a candidate
 - If its `Ref` bit is at zero, the selection is done
 - If not, the `Ref` bit of the candidate frame is reset, the frame is inserted again in the FIFO, and the process proceeds with the next frame
 - The process ends when a frame with the `Ref` bit at zero is found
- Note that such frame is always found

Page replacement policies

Clock algorithm

- The **clock** policy is an improvement of the second chance algorithm, avoiding the removal and reinsetion of elements in the FIFO
- The list is transformed into a circular one and a pointer signals the oldest element
 - The action of removal followed by a reinsetion corresponds to a pointer advance
- On page replacement:
 - While the `Ref` bit of a frame is non-zero, that bit is reset and the pointer advances to the next frame
 - The first frame with the `Ref` bit at zero is chosen for replacement
 - After replacement, the pointer is placed pointing to the next element



Working set

- Assume that initially only 2 pages of a process are in memory
 - The one containing the first instruction
 - The one containing the start of the stack
- After execution starts and for a while, page faults will be frequent
- Then the process will enter a phase in which page faults will be almost inexistent
 - Corresponds to a period where, accordingly to the principle of locality of reference, the fraction of the address space that the process is currently referencing is all present in main memory
- This set of pages is called the **working set** of the process
- Over time the working set of the process will vary, not only with respect to the number, but also with the specific pages that define it

Thrashing

- Consider that the maximum number of frames assigned to a process is fixed
- If this number is always greater or equal to the number of pages of the different working sets of the process:
 - the process's life will be a succession of periods with frequent page faults with periods almost without them
- If it is lower
 - the process will be continuously generating page faults
 - in such cases, it is said to be in **thrashing**
- Keeping the working set of a process always in memory is a page replacement design challenge

Demand paging vs. prepaging

- When a process transition to the ready state, what pages should be placed in main memory?
- Two possible strategies: **demand paging** and **prepaging**
- **Demand paging** – place none and wait for the page faults
 - inefficient
- **Prepaging** – place those most likely to be referenced
 - first time, the two pages mentioned before (code and stack)
 - next times, those that were in main memory when the process was suspended
 - more efficient

Bibliography

- Operating Systems: Internals and Design Principles, W. Stallings, Prentice-Hall International Editions, 7th Ed, 2012
 - Chapter 7: Memory Management (sections 7.1 to 7.4)
 - Chapter 8: Virtual Memory (sections 8.1 to 8.2)
- Operating Systems Concepts, A. Silberschatz, P. Galvin and G. Gagne, John Wiley & Sons, 9th Ed, 2013
 - Chapter 8: Main Memory (sections 8.1 to 8.5)
 - Chapter 9: Virtual Memory (sections 9.1 to 9.6)
- Modern Operating Systems, A. Tanenbaum and H. Bos, Pearson Education Limited, 4th Ed, 2015
 - Chapter 3: Memory Management (section 3.1 to 3.7)